

9IADT Tech Challenge Fase 1

Relatório Técnico-Acadêmico

Curso: Pós-Graduação em Inteligência Artificial para Desenvolvedores

Instituição: FIAP

Autor: Samuel Camilo

Resumo

Este trabalho apresenta o desenvolvimento e a avaliação de modelos preditivos de classificação supervisionada aplicados a dois cenários da área da saúde: predição de diabetes e classificação de câncer de mama. Foram utilizados pipelines de Machine Learning abrangendo pré-processamento, treinamento e avaliação, seguindo boas práticas acadêmicas e metodológicas.

1 Introdução

A aplicação de técnicas de aprendizagem de máquina na área da saúde permite a identificação de padrões complexos em dados clínicos. Este trabalho tem como objetivo desenvolver e comparar modelos de classificação binária, analisando seus desempenhos e limitações.

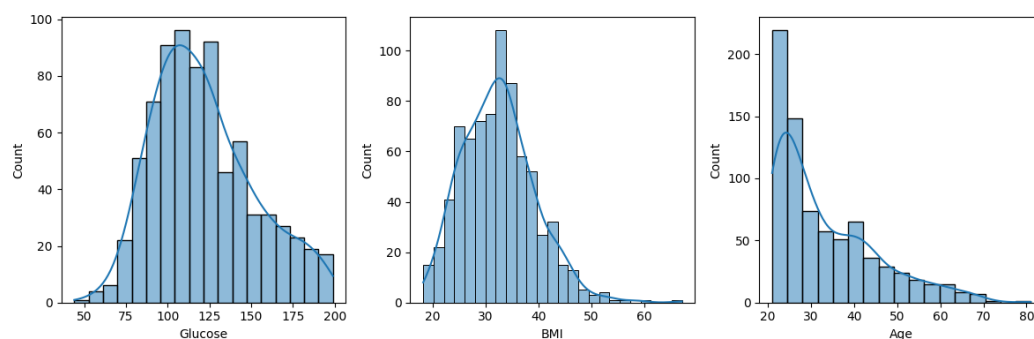
2 Metodologia

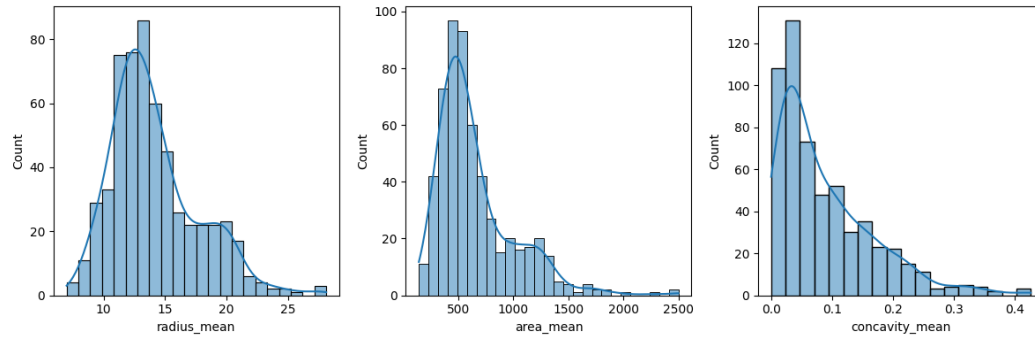
Foram utilizados dois conjuntos de dados públicos disponibilizados pela plataforma Kaggle: o dataset de Diabetes e o Breast Cancer Wisconsin Dataset.

2.1 Pré-processamento dos Dados

No dataset de Diabetes, valores zero em variáveis fisiologicamente implausíveis foram tratados como dados ausentes e imputados pela mediana. No Breast Cancer Wisconsin Dataset, colunas irrelevantes foram removidas e a variável alvo foi codificada de forma binária.

Figura 1 – Distribuição das principais variáveis dos datasets analisados.



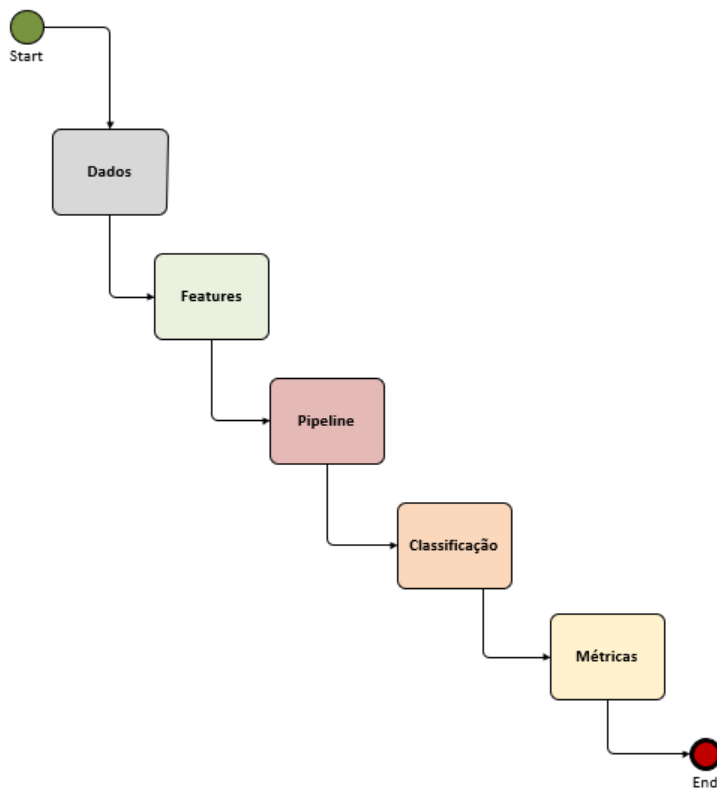


Fonte: Notebooks.

2.2 Modelos de Classificação

Foram avaliados dois modelos para cada cenário: Regressão Logística como baseline interpretável e modelos não lineares, tais como Random Forest e Support Vector Machine, com o objetivo de capturar relações mais complexas entre as variáveis.

Figura 2 – Estrutura do pipeline de Machine Learning adotado.

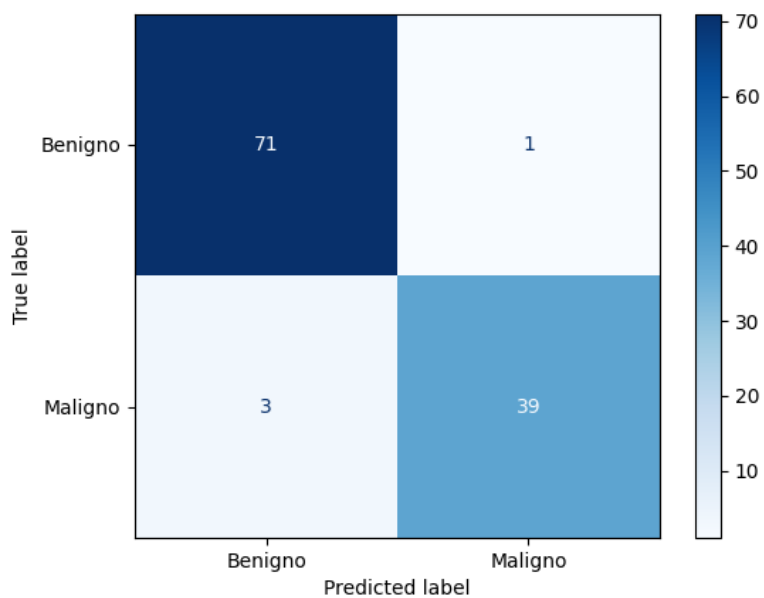
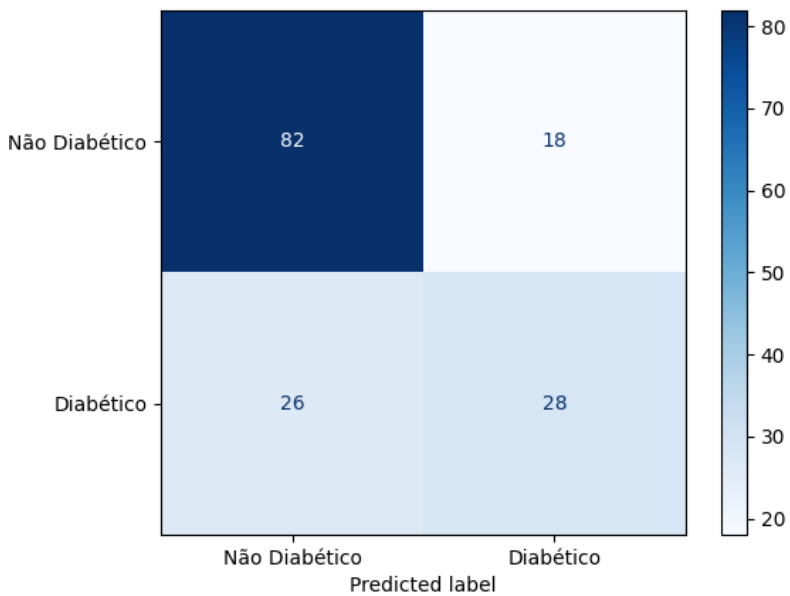


Fonte: Elaboração própria.

3 Resultados

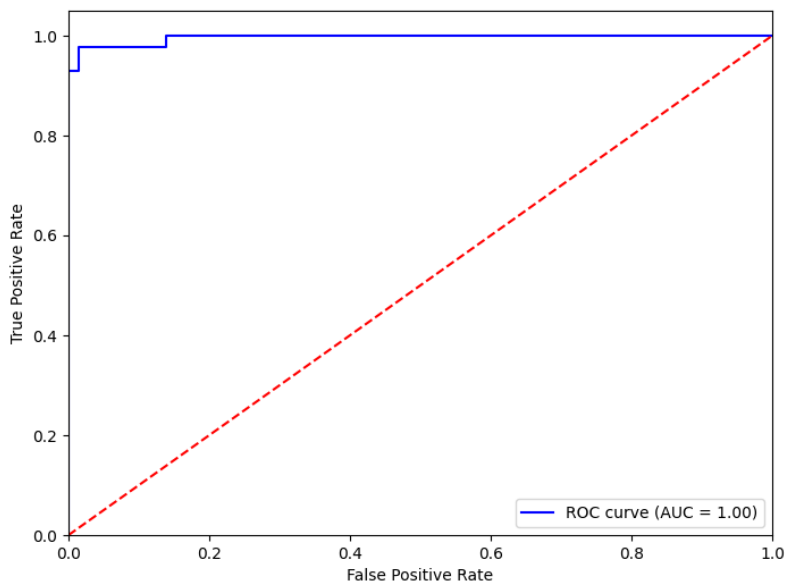
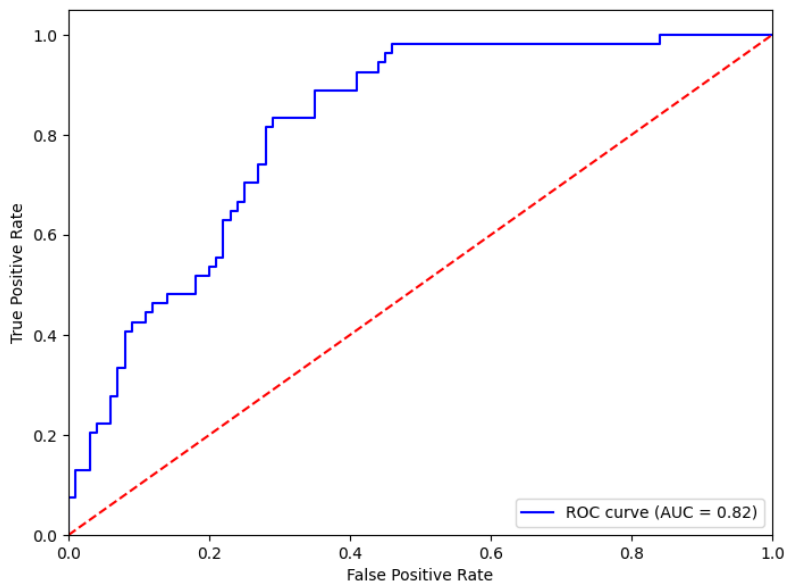
Os resultados indicaram elevado desempenho no Breast Cancer Wisconsin Dataset, com área sob a curva ROC igual a 1,0. Para o problema de Diabetes, observou-se maior complexidade e sobreposição entre classes, refletida em AUC aproximada de 0,82.

Figura 3 – Matrizes de confusão obtidas para os datasets de Diabetes e Breast Cancer Wisconsin.



Fonte: Notebooks.

Figura 4 – Curvas ROC dos modelos de classificação aplicados aos datasets analisados.



Fonte: Notebooks.

4 Discussão

Os resultados evidenciam a diferença de complexidade entre os problemas estudados. Enquanto o Breast Cancer Wisconsin Dataset apresenta alta separabilidade entre classes,

o dataset de Diabetes envolve maior ruído e sobreposição, o que explica o desempenho inferior obtido.

5 Conclusão

Conclui-se que os modelos desenvolvidos são adequados como baselines acadêmicos e para fins educacionais. Todavia, aplicações práticas reais exigem validações adicionais, análise clínica especializada e monitoramento contínuo.

Referências

KAGGLE. Breast Cancer Wisconsin Dataset. Disponível em:

<https://www.kaggle.com/datasets/mathchi/diabetes-data-set>

KAGGLE. Diabetes Dataset. Disponível em:

<https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

Repositório GitHub:

https://github.com/scamilo77/9IADT_TechChallenge_1

Apêndices - Códigos Fonte:

- a) README.md
- b) data.py
- c) features.py
- d) pipelines.py
- e) train.py
- f) eda_breast_cancer.ipynb
- g) eda_diabetes.ipynb
- h) Dockerfile

Apêndice a: README.md

Tech Challenge – Modelos Preditivos de Classificação

Este projeto implementa pipelines de Machine Learning para **classificação supervisionada** em dois cenários de saúde:

- Predição de Diabetes
- Classificação de Câncer de Mama (Breast Cancer Wisconsin)

O foco é demonstrar boas práticas de pré-processamento, pipelines, avaliação e reprodutibilidade.

Estrutura do Projeto

'''

.

```
|— data/
|   |— raw/
|— notebooks/
|   |— 01_eda_diabetes.ipynb
|   |— 02_eda_breast_cancer.ipynb
|— src/
|   |— project_name/
|       |— data.py
```



```
|   |— features.py
|   |— pipelines.py
|   └— train.py
|— Dockerfile
|— requirements.txt
└— README.md
```

```
'''
```

```
---
```

Datasets

- Breast Cancer Wisconsin: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
- Diabetes Dataset: <https://www.kaggle.com/datasets/mathchi/diabetes-data-set>

Os arquivos CSV devem ser baixados e colocados em `data/raw/`.

```
---
```

Execução Local

```
```bash
```

```
pip install -r requirements.txt
```

```
python src/project_name/train.py
```

```
'''
```

```

```

```
Execução com Docker
```

```
```bash
```

```
docker build -t techchallenge-ml .
```

```
docker run --rm techchallenge-ml
```

```
'''
```

```
---
```

```
## Execução com Jupyter Notebook
```

```
```bash
```

```
pip install jupyter notebook ipykernel
```

```
jupyter notebook
```

```
'''
```

Abra os notebooks da pasta `notebooks/` e selecione o kernel Python correto.

```

```

```
Modelos Utilizados
```

- Logistic Regression
- Random Forest (Diabetes)
- SVM (Breast Cancer)

---

## Observação

Este projeto é educacional/experimental e \*\*não deve ser usado para decisões clínicas reais\*\*.

Apêndice B – data.py

```
import os
```

```
from kaggle.api.kaggle_api_extended import KaggleApi
```

```
import pandas as pd
```

```
from pathlib import Path
```

```
RAW_DATA_DIR = "data/raw"
```

```
def download_diabetes_file():
```

```
 api = KaggleApi()
```

```
 api.authenticate()
```

```
 target_dir = os.path.join(RAW_DATA_DIR, "diabetes-data-set")
```

```
 os.makedirs(target_dir, exist_ok=True)
```

```
 api.dataset_download_files(
```

```
 "mathchi/diabetes-data-set",
```

```
 path=target_dir,
```

```
 unzip=True)
```

```
def load_diabetes_data():
```

```
download_diabetes_file()
```

```
file_path = os.path.join(RAW_DATA_DIR, "diabetes-data-set/diabetes.csv")
```

```
return pd.read_csv(file_path)
```

```
def download_breast_cancer_file():
```

```
 api = KaggleApi()
```

```
 api.authenticate()
```

```
 target_dir = os.path.join(RAW_DATA_DIR, "breast-cancer-wisconsin-data")
```

```
 os.makedirs(target_dir, exist_ok=True)
```

```
 api.dataset_download_files(
```

```
 "uciml/breast-cancer-wisconsin-data",
```

```
 path=target_dir,
```

```
 unzip=True)
```

```
def load_breast_cancer_data():
```

```
 download_breast_cancer_file()
```

```
 file_path = os.path.join(RAW_DATA_DIR, "breast-cancer-wisconsin-data/data.csv")
```

```
 return pd.read_csv(file_path)
```

Apêndice C – features.py

```
import numpy as np
```

```
import pandas as pd
```

```
from typing import Tuple
```

```
import pandas as pd
```

```
import os
```

```
PROCESSED_DATA_DIR = "data/processed"
```

```

```

```
Diabetes Dataset
```

```
#-----
```

```
DIABETES_ZERO_AS_MISSING = [
```

```
 "Glucose",
```

```
 "BloodPressure",
```

```
 "SkinThickness",
```

```
 "Insulin",
```

```
 "BMI"
```

```
]
```

```
def clean_diabetes_data(df: pd.DataFrame) -> pd.DataFrame:
```

```

if not isinstance(df, pd.DataFrame):
 raise TypeError("Input must be a pandas DataFrame.")

df = df.copy(deep=True)

Substitui zeros por NaN nas colunas relevantes
for col in DIABETES_ZERO_AS_MISSING:
 df[col] = df[col].replace(0, np.nan)

Preenche NaN com a mediana da coluna
for col in DIABETES_ZERO_AS_MISSING:
 median_value = df[col].median()
 df[col].fillna(median_value, inplace=True)

dir = os.path.join(PROCESSED_DATA_DIR, "diabetes-data-set")
os.makedirs(dir, exist_ok=True)

df.to_csv(os.path.join(PROCESSED_DATA_DIR, "diabetes-data-set", "diabetes.csv"),
index=False)

return df

def split_diabetes_data(df: pd.DataFrame, target_column: str) -> Tuple[pd.DataFrame,
pd.Series]:
 X = df.drop(columns=["Outcome"])
 y = df["Outcome"]
 return X, y

```

```

Breast Cancer Dataset
#-----

def clean_breast_cancer_data(df: pd.DataFrame) -> pd.DataFrame:

 df = df.copy(deep=True)

 # Remove colunas irrelevantes se existirem

 cols_to_drop = ["id", "Unnamed: 32"]

 df.drop(columns=[c for c in cols_to_drop if c in df.columns], inplace=True)

 # Converte diagnóstico para binário

 df["diagnosis"] = df["diagnosis"].map({"M": 1, "B": 0})

 dir = os.path.join(PROCESSED_DATA_DIR, "breast-cancer-wisconsin-data")

 os.makedirs(dir, exist_ok=True)

 df.to_csv(os.path.join(PROCESSED_DATA_DIR, "breast-cancer-wisconsin-data",
"data.csv"), index=False)

 return df

def split_breast_cancer_data(df: pd.DataFrame, target_column: str) ->
Tuple[pd.DataFrame, pd.Series]:

 X = df.drop(columns=["diagnosis"])

 y = df["diagnosis"]

 return X, y

```



Apêndice D – pipelines.py

```
from sklearn.pipeline import Pipeline

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.svm import SVC
```

```
def build_diabetes_pipeline():

 pipeline = Pipeline(

 steps=[

 ('scaler', StandardScaler()),

 ('model',

 LogisticRegression(

 max_iter=1000,

 random_state=42,

 solver='liblinear'

))

]

)

 return pipeline
```

```
def build_diabetes_pipeline_rf():

 pipeline = Pipeline(

 steps=[

 ('model',

 RandomForestClassifier(
```

```
 n_estimators=200,
 random_state=42,
 max_depth=None,
 min_samples_split=5
))
]
return pipeline
```

```
def build_breast_cancer_pipeline():
```

```
 pipeline = Pipeline(
 steps=[
 ('scaler', StandardScaler()),
 ('model',
 LogisticRegression(
 max_iter=2000,
 random_state=42,
 solver='lbfgs'
))
]
)
 return pipeline
```

```
def build_breast_cancer_pipeline_rf():
```

```
 pipeline = Pipeline(
 steps=[
```

```
 ('scaler', StandardScaler()),
 ('model',
 SVC(
 C=1.0,
 kernel='rbf',
 random_state=42,
 gamma='scale',
 probability=True
))
])
return pipeline
```

## Apêndice E – train.py

```
from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report

from data import load_diabetes_data, load_breast_cancer_data

from features import clean_diabetes_data, split_diabetes_data, clean_breast_cancer_data,
split_breast_cancer_data

from pipelines import build_diabetes_pipeline, build_breast_cancer_pipeline,
build_diabetes_pipeline_rf, build_breast_cancer_pipeline_rf

def train_diabetes_model():

 diabetes_data = load_diabetes_data()

 diabetes_data = clean_diabetes_data(diabetes_data)

 X, y = split_diabetes_data(diabetes_data, target_column="Outcome")

 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

 models = {

 "Logistic Regression": build_diabetes_pipeline(),

 "Random Forest": build_diabetes_pipeline_rf()

 }

 for model_name, pipeline in models.items():

 pipeline.fit(X_train, y_train)

 y_pred = pipeline.predict(X_test)

 print(f'{model_name} Diabetes Accuracy: {accuracy_score(y_test, y_pred)}")
```

```
 print(f'{model_name} Diabetes Classification Report:\n',
classification_report(y_test, y_pred))
```

```
def train_breast_cancer_model():
```

```
 breast_cancer_data = load_breast_cancer_data()
```

```
 breast_cancer_data = clean_breast_cancer_data(breast_cancer_data)
```

```
 X, y = split_breast_cancer_data(breast_cancer_data, target_column="diagnosis")
```

```
 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
 models = {
```

```
 "Logistic Regression": build_breast_cancer_pipeline(),
```

```
 "SVM": build_breast_cancer_pipeline_rf()
```

```
 }
```

```
 for model_name, pipeline in models.items():
```

```
 pipeline.fit(X_train, y_train)
```

```
 y_pred = pipeline.predict(X_test)
```

```
 print(f'{model_name} Breast Cancer Accuracy: {accuracy_score(y_test, y_pred)}')
```

```
 print(f'{model_name} Classification Report:\n', classification_report(y_test,
y_pred))
```

```
if __name__ == "__main__":
```

```
 train_diabetes_model()
```

```
 train_breast_cancer_model()
```

Apêndice f - eda\_breast\_cancer.ipynb

```
import os
```

```
import sys
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
module_path = os.path.abspath(os.path.join('..','src'))
```

```
if module_path not in sys.path:
```

```
 sys.path.append(module_path)
```

```
from DiagnosisTool.features import clean_breast_cancer_data
```

```
module_path = os.path.abspath(os.path.join('..',module_path,'DiagnosisTool'))
```

```
if module_path not in sys.path:
```

```
 sys.path.append(module_path)
```

```
from data import download_breast_cancer_file
```

```
from data import load_breast_cancer_data
```

```
download_breast_cancer_file()
```

```
breast_cancer_data = load_breast_cancer_data()
```

```
breast_cancer_data = clean_breast_cancer_data(breast_cancer_data)
```

```
breast_cancer_data.describe()
```

Dataset URL: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

Dataset URL: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

## Figuras

### Figura 1 - Distribuição de variáveis

A figura 1 apresenta as distribuições das principais variáveis utilizadas na modelagem, permitindo avaliar assimetria, presença de outliers e justificar as decisões de pré-processamento aplicadas

```
cols = ["radius_mean", "area_mean", "concavity_mean"]
```

```
plt.figure(figsize=(12, 4))
```

```
for i, col in enumerate(cols):
```

```
 plt.subplot(1, 3, i + 1)
```

```
 sns.histplot(breast_cancer_data[col], kde=True)
```

```
 plt.title = f'Distribution of {col}'
```

```
plt.tight_layout()
```

```
os.makedirs("figures", exist_ok=True)
```

```
plt.savefig("figures/breast_cancer_data_distributions.png")
```

```
plt.show()
```

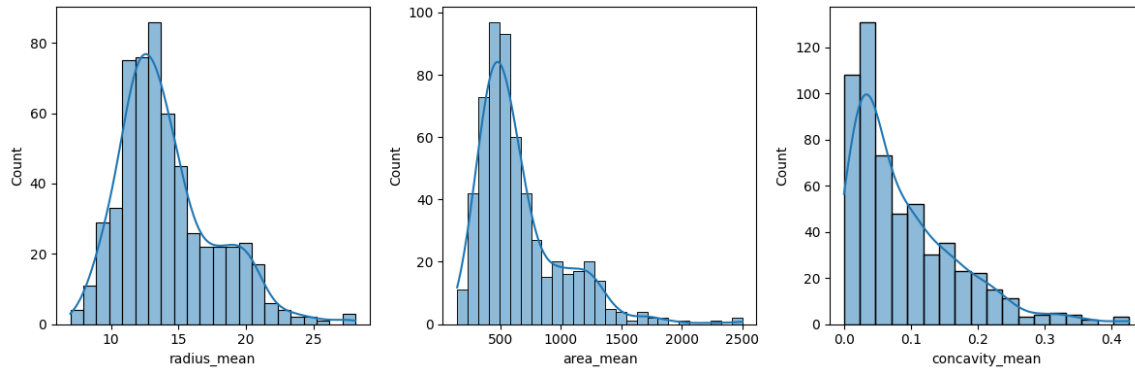


Figura 2 - Pipeline de Machine Learning

O diagrama do Pipeline de Machine Learning foi elaborado manualmente para representar as etapas do processo de Machine Learning, desde a entrada dos dados até a avaliação dos modelos

Figura 3 - Matriz de Confusão (Breast Cancer)

A matriz de confusão evidencia a separabilidade entre tumores benignos e malignos obtida pelo modelo treinado

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

from DiagnosisTool.data import load_breast_cancer_data

from DiagnosisTool.features import clean_breast_cancer_data, split_breast_cancer_data

from sklearn.model_selection import train_test_split

from pipelines import build_breast_cancer_pipeline

df = load_breast_cancer_data()

df = clean_breast_cancer_data(df)

df.head()

X, y = split_breast_cancer_data(df, "Outcome")
```



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

```
pipeline = build_breast_cancer_pipeline()
```

```
pipeline.fit(X_train, y_train)
```

```
y_pred = pipeline.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Benigno',
'Maligno'])
```

```
disp.plot(cmap=plt.cm.Blues)
```

```
plt.title = "Confusion Matrix - Breast Cancer Wisconsin Data"
```

```
os.makedirs("figures", exist_ok=True)
```

```
plt.savefig("figures/breast_cancer_confusion_matrix.png")
```

```
plt.show()
```

Dataset URL: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

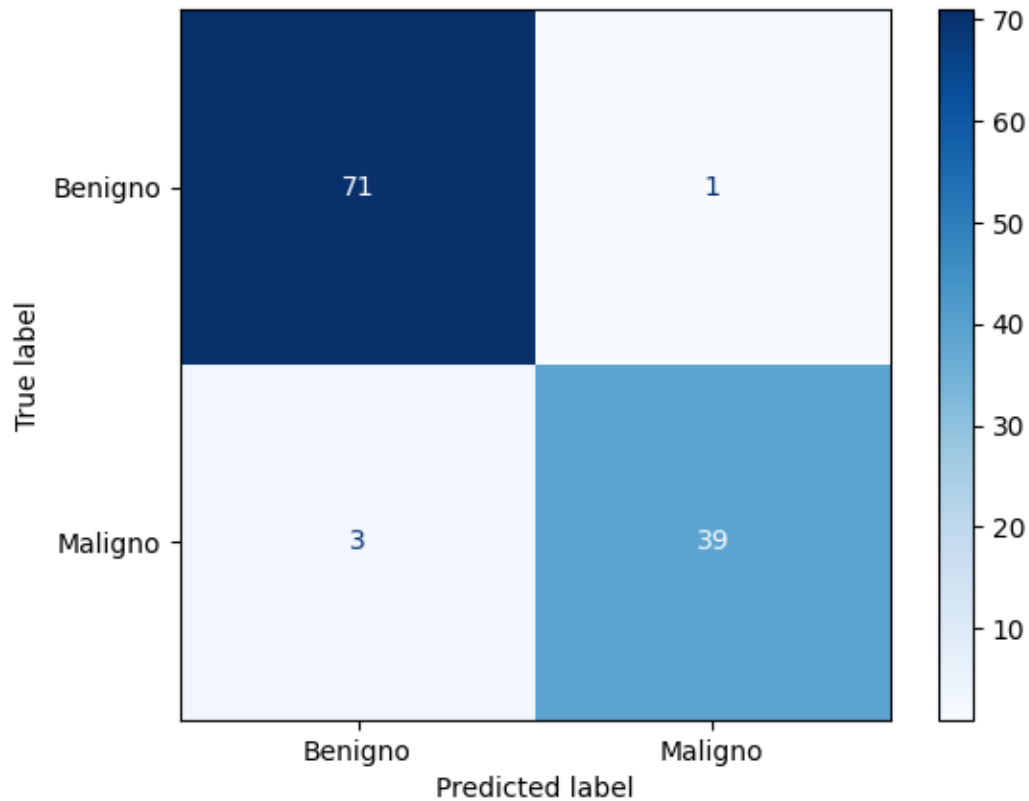


Figura 4 - Curvas ROC

A figura 4 apresenta as curvas ROC (Receiver Operating Characteristics) obtidas para os modelos de classificação aplicados aos datasets analisados. Essa métrica avalia a capacidade discriminativa dos modelos independentemente do limiar de decisão, sendo especialmente relevante em problemas de classificação binária. A área sob a curva ROC (AUC) indica o desempenho global do modelo, onde valores mais próximos de 1 representam maior capacidade de separação entre as classes.

```
from sklearn.metrics import roc_curve, auc
```

```
y_prob = pipeline.predict_proba(X_test)[:, 1]
```

```
fpr, tpr, _ = roc_curve(y_test, y_prob)
```

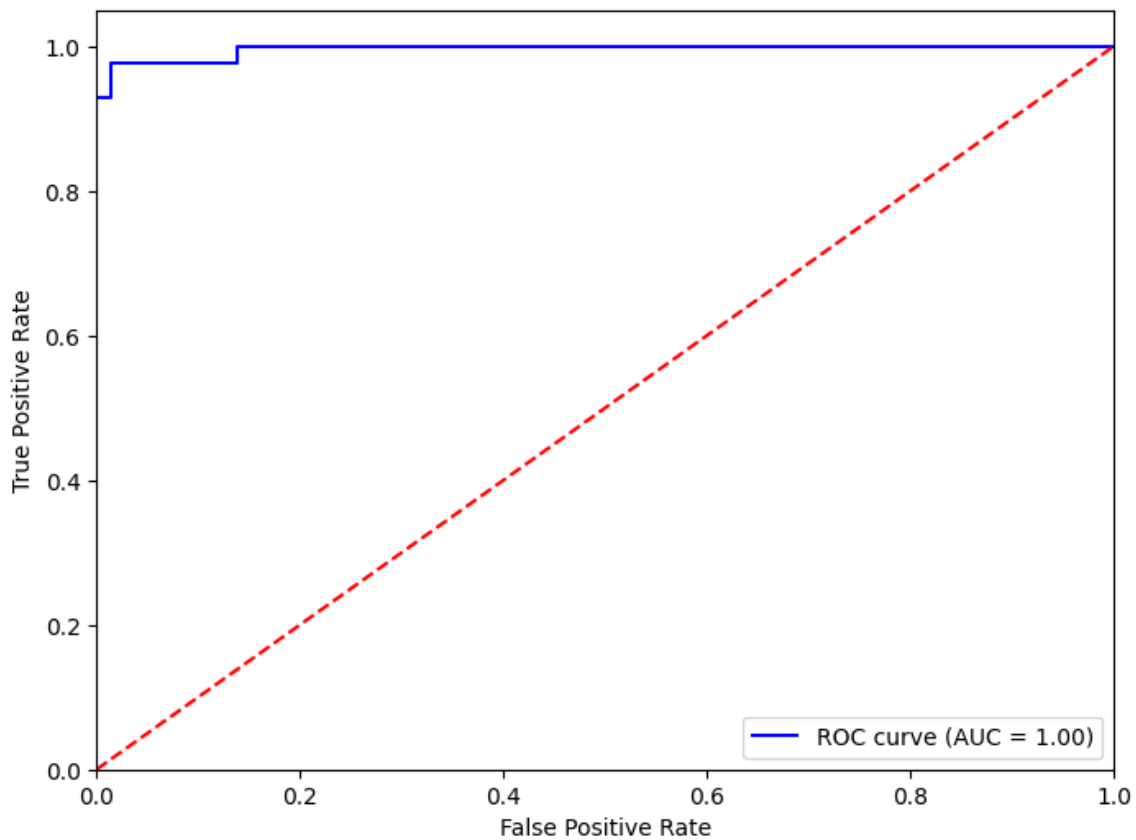
```
roc_auc = auc(fpr, tpr)
```

```
plt.figure(figsize=(8, 6))
```

```

plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title = 'ROC Curve - Breast Cancer Wisconsin Data'
plt.legend(loc="lower right")
os.makedirs("figures", exist_ok=True)
plt.savefig("figures/breast_cancer_roc_curve.png")
plt.show()

```



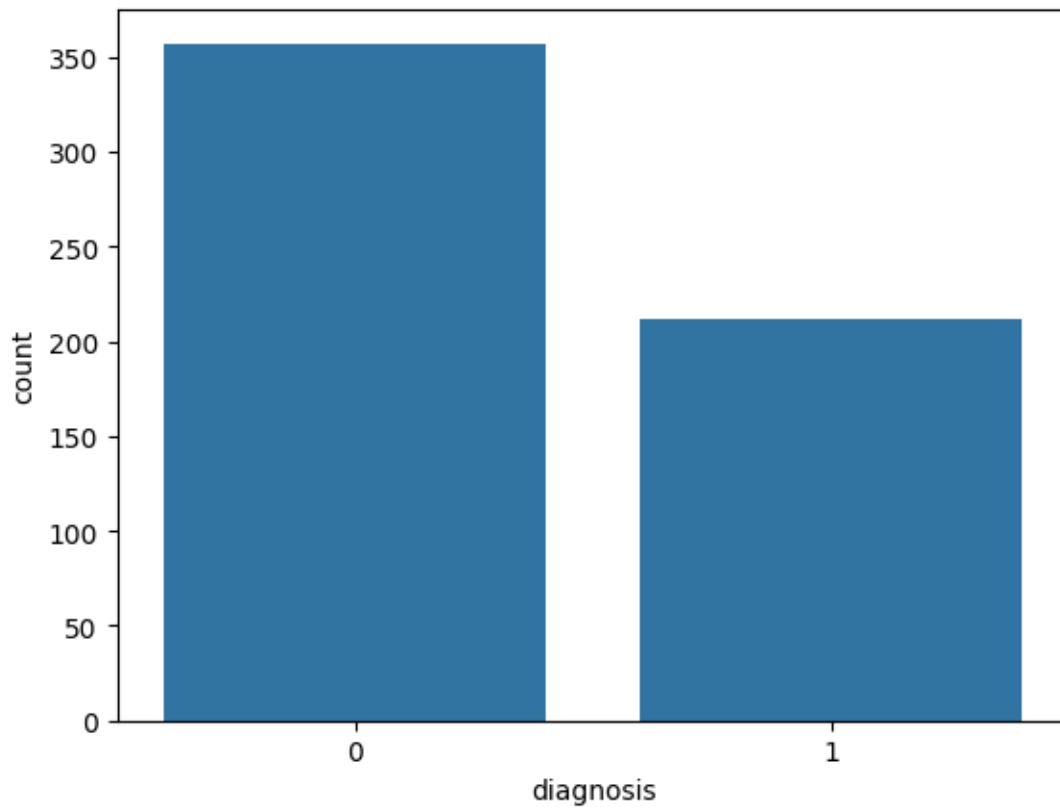
```

sns.countplot(x="diagnosis", data=breast_cancer_data)

```

```
plt.title="Distribuição das Classes de Diagnóstico (0: Benigno, 1: Maligno)"
```

```
plt.show()
```



## Discussões

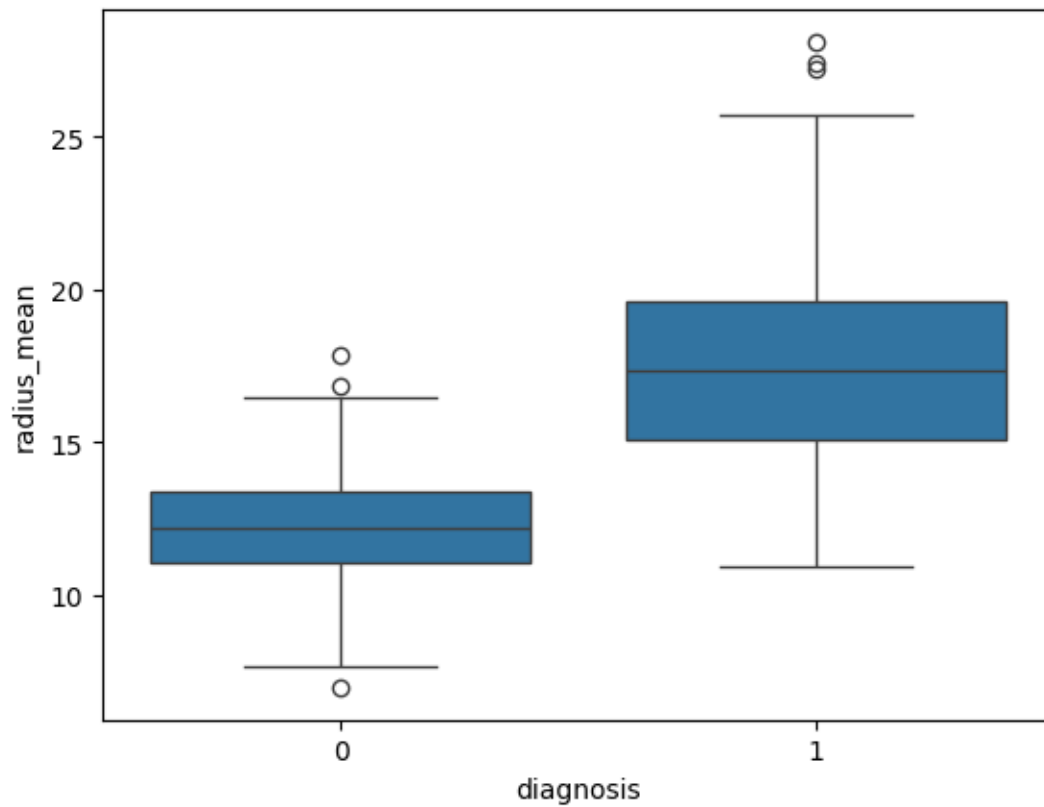
### Discussão da variável alvo (diagnóstico)

- Dataset moderadamente balanceado
- Leve predominância de tumores benignos
- Métricas além da eficiência são recomendadas

```
sns.boxplot(x="diagnosis",y="radius_mean",data=breast_cancer_data)
```

```
plt.title="Radius por Classe de Diagnóstico"
```

```
plt.show()
```



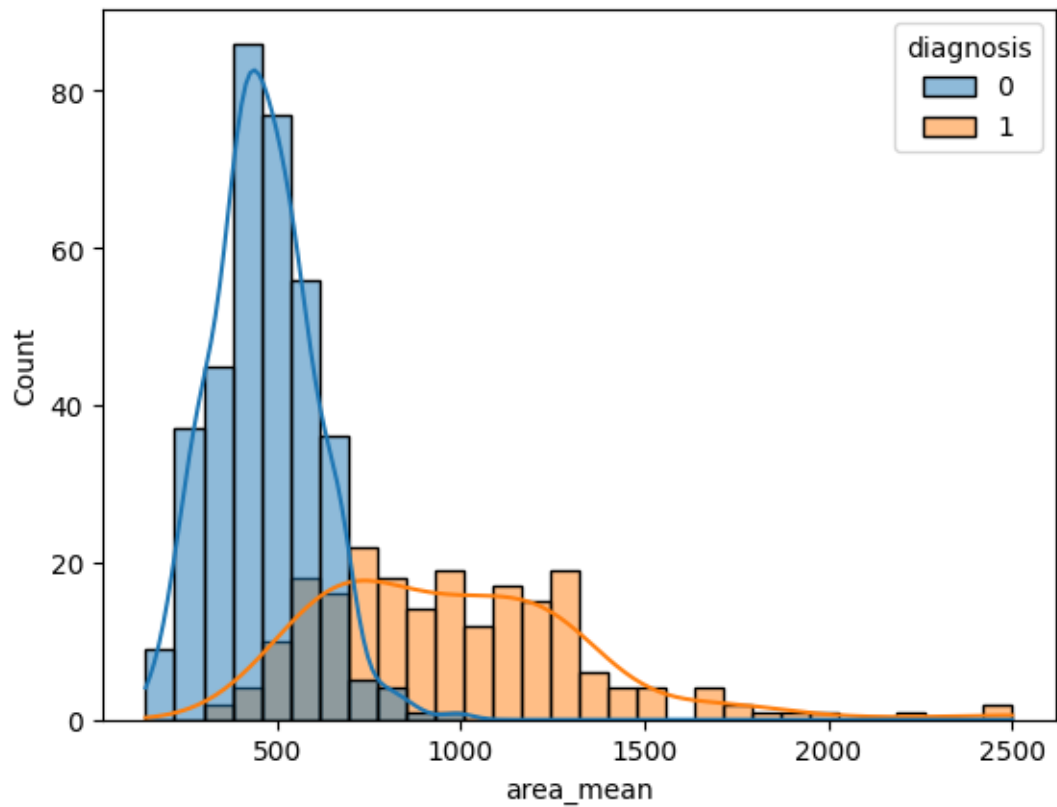
Discussão de Radius por classe

- Tumores malignos apresentam raio (radius) médio maior
- Boa separabilidade visual
- Feature altamente discriminante

```
sns.histplot(data=breast_cancer_data, x="area_mean", hue="diagnosis", kde=True, bins=30)
```

```
plt.title="Distribuição de Área por Classe"
```

```
plt.show()
```



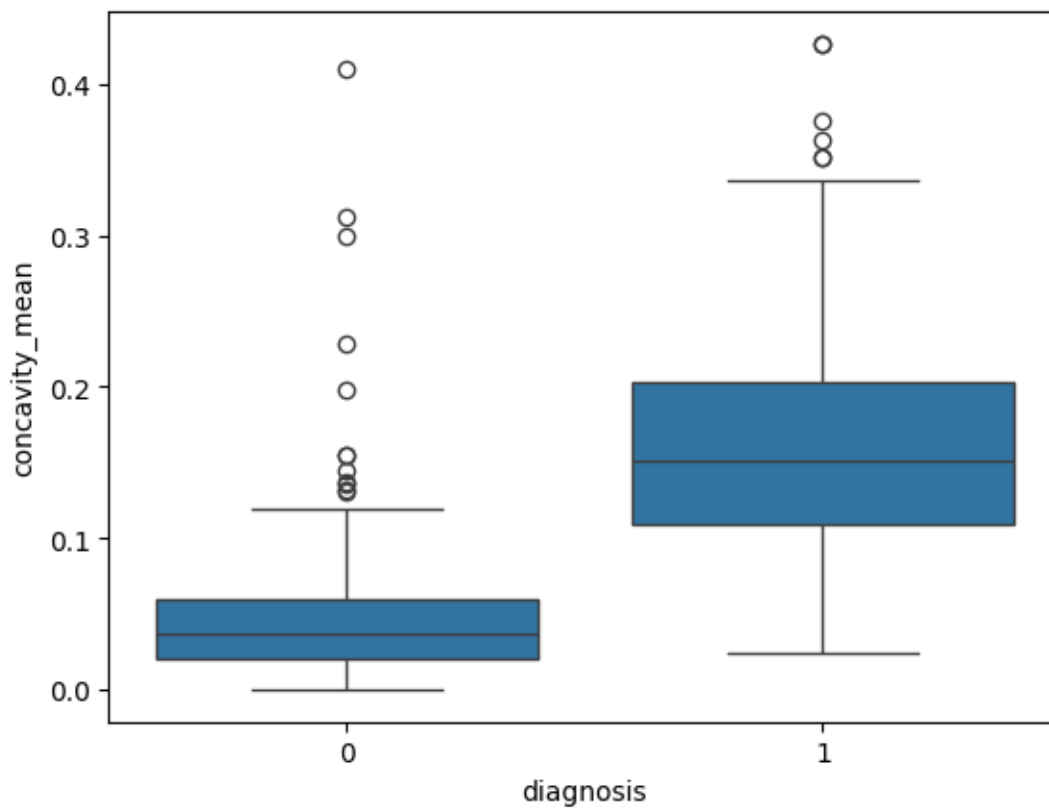
#### Discussão de Área por Classe

- Distribuição dos malignos deslocada à direita
- Overlap limitado

```
sns.boxplot(x="diagnosis", y="concavity_mean", data=breast_cancer_data)
```

```
plt.title="Distribuição de Concavidade por Classe"
```

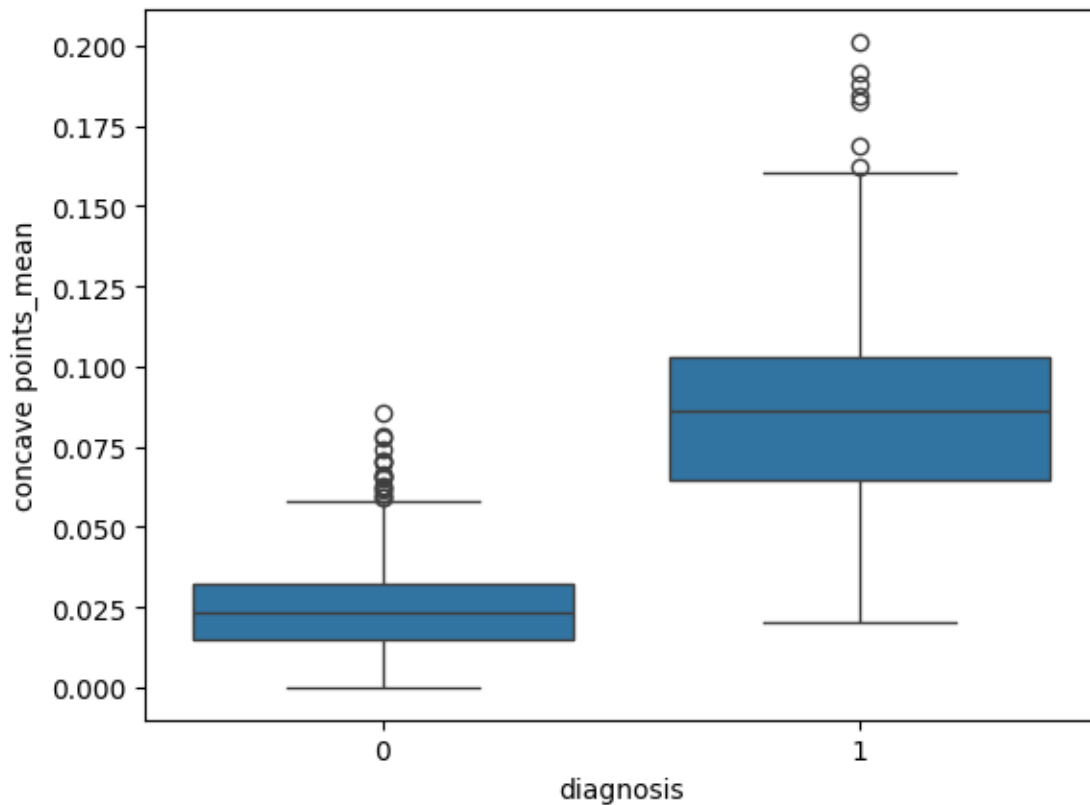
```
plt.show()
```



```
sns.boxplot(x="diagnosis", y="concave points_mean", data=breast_cancer_data)
```

```
plt.title="Distribuição de Pontos Concavos por Classe"
```

```
plt.show()
```



#### Discussão sobre Concavity e Concave Points

- Tumores malignos apresentam valores significativamente maiores
- Bordas mais irregulares (maior agressividade tumoral)
- Algumas das features mais importantes do Dataset

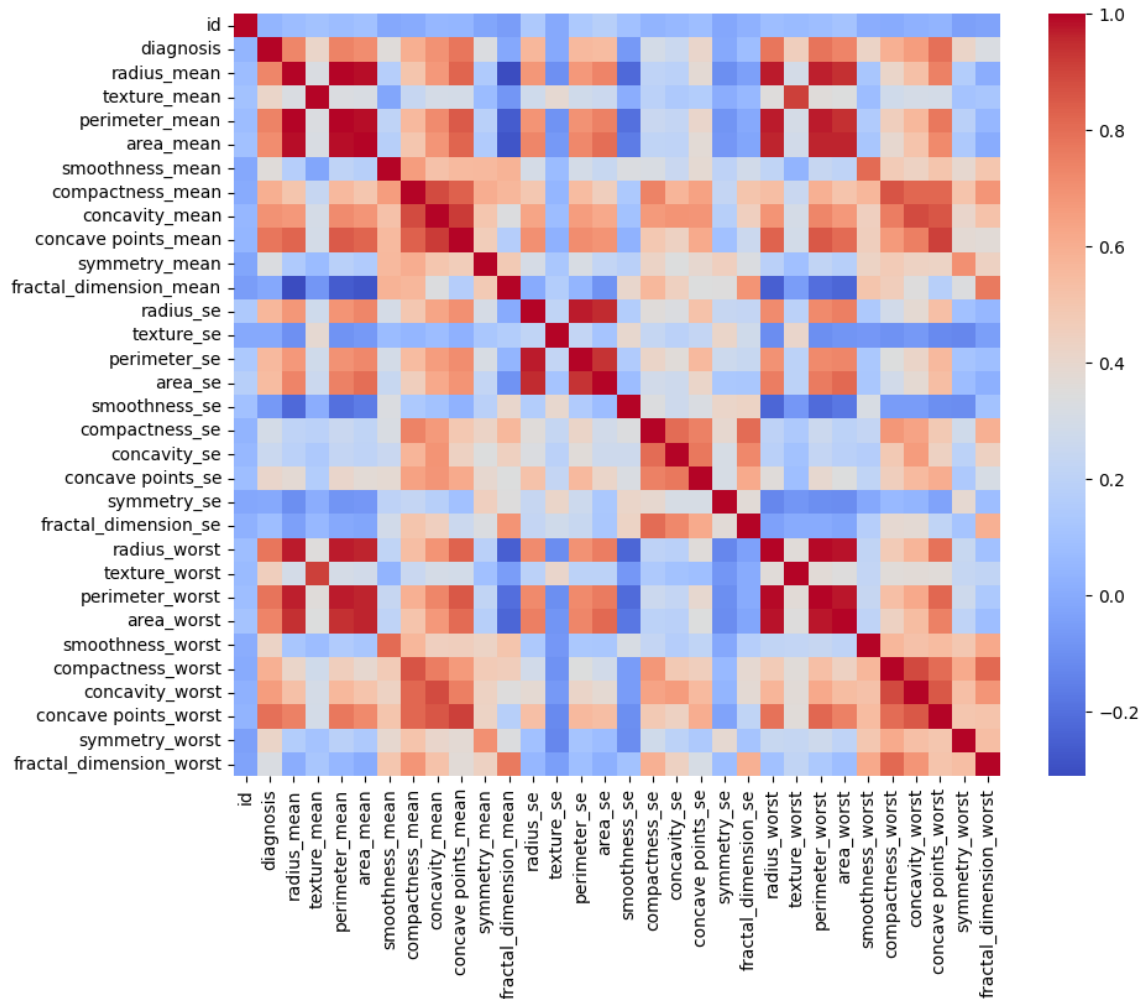
```
plt.figure(figsize=(10,8))
```

```
breast_cancer_data_numeric = breast_cancer_data.select_dtypes(include=['float64',
'int64'])
```

```
sns.heatmap(breast_cancer_data_numeric.corr(), cmap="coolwarm" , annot=False)
```

```
plt.show()
```





## Observações

- Forte correlação entre:
  - Radius, perimeter & area
  - Concavity & Concave Points
- Multicolinearidade presente (modelos lineares podem exigir regularização)

## Discussão geral dos resultados

### Principais insights

- Features geométricas são altamente discriminativas
  - Radius, perimeter e area apresentam forte separação entre as classes
- Irregularidade de borda é fator-chave

- Concavity e Concave Points são ótimos indicadores de malignidade
- Features isoladas não bastam
  - Variáveis como texture e smoothness têm baixo poder isoladamente, mas são valiosas em conjunto
- Dataset é limpo e bem estruturado
  - Não há ausência de valores

Apêndice G - eda\_diabetes.ipynb

```
import os
```

```
import sys
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
module_path = os.path.abspath(os.path.join('..','src'))
```

```
if module_path not in sys.path:
```

```
 sys.path.append(module_path)
```

```
from DiagnosisTool.features import clean_diabetes_data
```

```
from DiagnosisTool.pipelines import build_diabetes_pipeline
```

```
module_path = os.path.abspath(os.path.join(module_path,'DiagnosisTool'))
```

```
if module_path not in sys.path:
```

```
 sys.path.append(module_path)
```

```
from data import download_diabetes_file
```

```
from data import load_diabetes_data
```

```
download_diabetes_file()
```

```
diabetes_data = load_diabetes_data()
```

```
diabetes_data = clean_diabetes_data(diabetes_data)
```

```
diabetes_data.describe()
```

Dataset URL: <https://www.kaggle.com/datasets/mathchi/diabetes-data-set>

Dataset URL: <https://www.kaggle.com/datasets/mathchi/diabetes-data-set>

### Figura 1 - Distribuição de variáveis

A figura 1 apresenta as distribuições das principais variáveis utilizadas na modelagem, permitindo avaliar assimetria, presença de outliers e justificar as decisões de pré-processamento aplicadas

```
cols = ["Glucose", "BMI", "Age"]
```

```
plt.figure(figsize=(12, 4))
```

```
for i, col in enumerate(cols):
```

```
 plt.subplot(1, 3, i + 1)
```

```
 sns.histplot(diabetes_data[col], kde=True)
```

```
 plt.title = f"Distribution of {col}"
```

```
plt.tight_layout()
```

```
os.makedirs("figures", exist_ok=True)
```

```
plt.savefig("figures/diabetes_data_distributions.png")
```

```
plt.show()
```

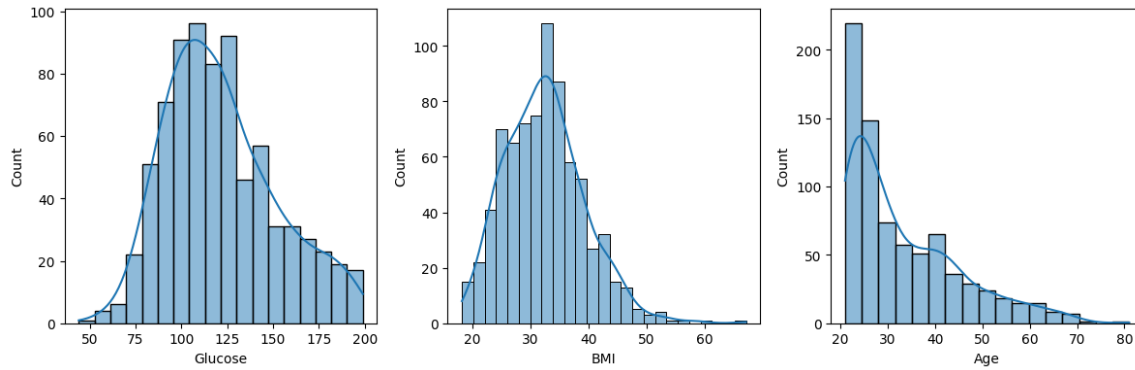


Figura 2 - Pipeline de Machine Learning

O diagrama do Pipeline de Machine Learning foi elaborado manualmente para representar as etapas do processo de Machine Learning, desde a entrada dos dados até a avaliação dos modelos

Figura 3 - Matriz de Confusão

A matriz de confusão permite avaliar a distribuição de acertos e erros do modelo de classificação aplicado ao dataset de Diabetes

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
from DiagnosisTool.data import load_diabetes_data
```

```
from DiagnosisTool.features import clean_diabetes_data, split_diabetes_data
```

```
from sklearn.model_selection import train_test_split
```

```
df = load_diabetes_data()
```

```
df = clean_diabetes_data(df)
```

```
df.head()
```

```
X, y = split_diabetes_data(df, "Outcome")
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

```
pipeline = build_diabetes_pipeline()
```

```
pipeline.fit(X_train, y_train)
```

```
y_pred = pipeline.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Não Diabético',
'Diabético'])
```

```
disp.plot(cmap=plt.cm.Blues)
```

```
plt.title = "Confusion Matrix - Diabetes"
```

```
os.makedirs("figures", exist_ok=True)
```

```
plt.savefig("figures/diabetes_confusion_matrix.png")
```

```
plt.show()
```

Dataset URL: <https://www.kaggle.com/datasets/mathchi/diabetes-data-set>

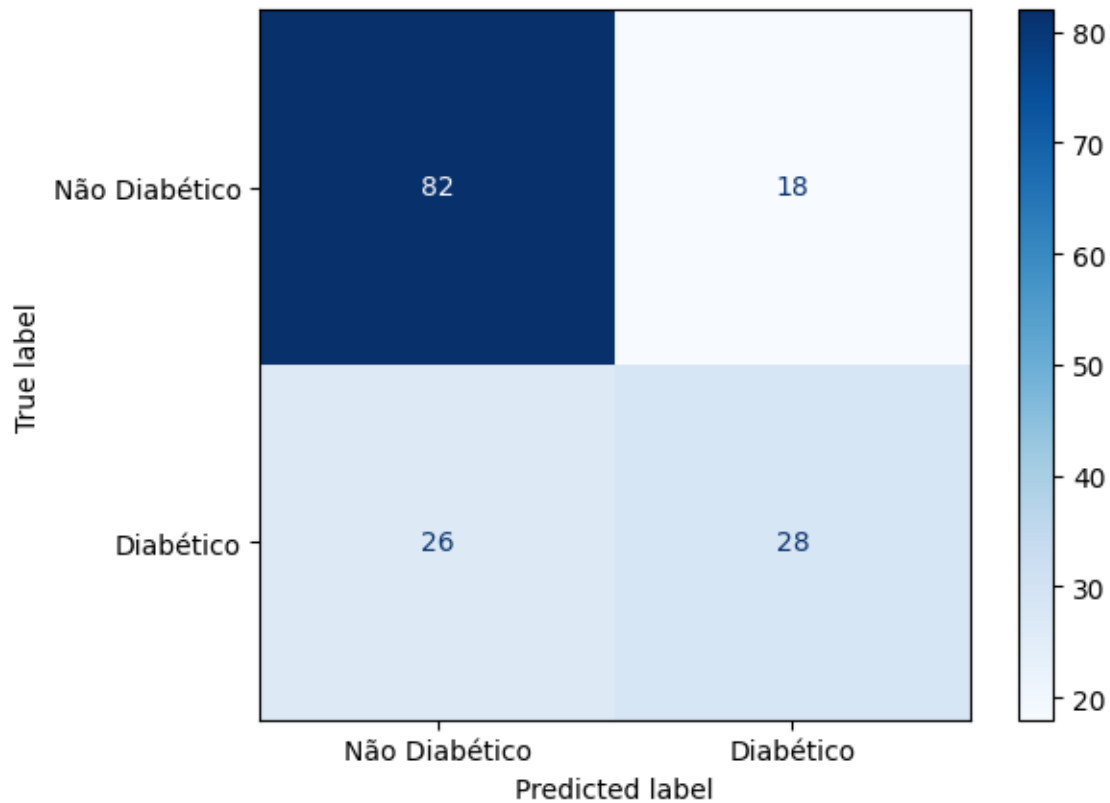


Figura 4 - Curvas ROC

A figura 4 apresenta as curvas ROC (Receiver Operating Characteristics) obtidas para os modelos de classificação aplicados aos datasets analisados. Essa métrica avalia a capacidade discriminativa dos modelos independentemente do limiar de decisão, sendo especialmente relevante em problemas de classificação binária. A área sob a curva ROC (AUC) indica o desempenho global do modelo, onde valores mais próximos de 1 representam maior capacidade de separação entre as classes.

```
from sklearn.metrics import roc_curve, auc
```

```
y_prob = pipeline.predict_proba(X_test)[:, 1]
```

```
fpr, tpr, _ = roc_curve(y_test, y_prob)
```

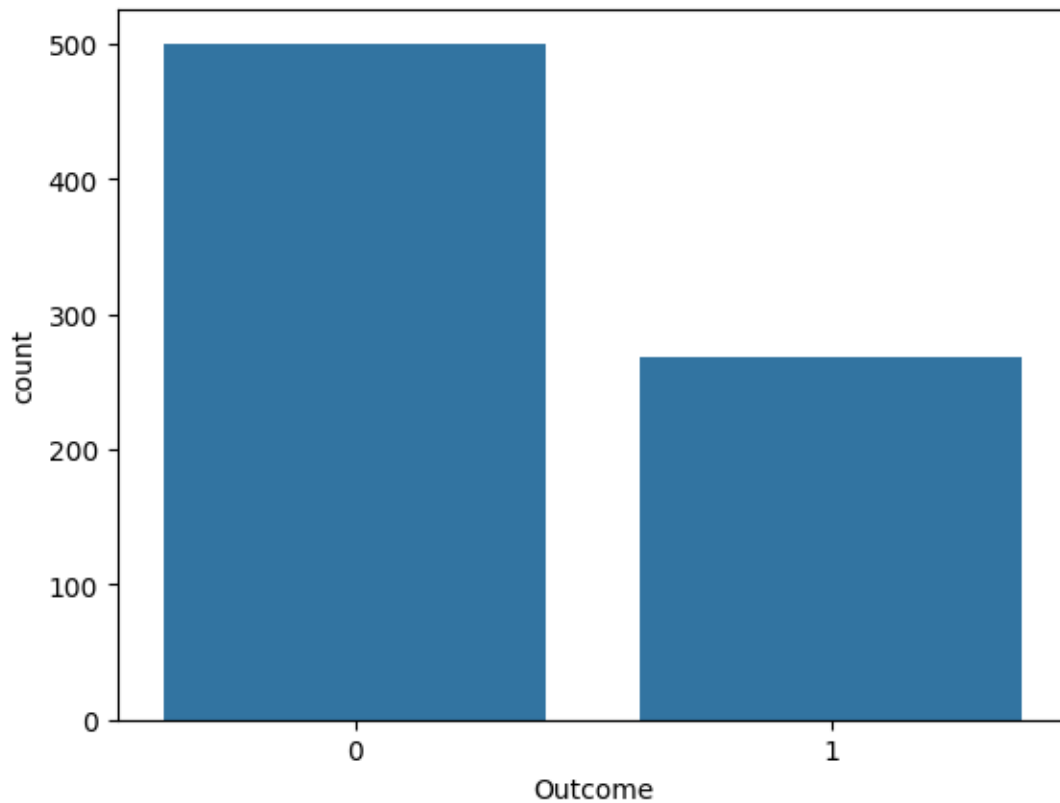
```
roc_auc = auc(fpr, tpr)
```

```
plt.figure(figsize=(8, 6))
```

```
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title = 'ROC Curve - Diabetes Data'
plt.legend(loc="lower right")
os.makedirs("figures", exist_ok=True)
plt.savefig("figures/diabetes_roc_curve.png")
plt.show()

sns.countplot(x="Outcome", data=diabetes_data)
plt.title="Distribuição da variável Outcome"
plt.show()
```





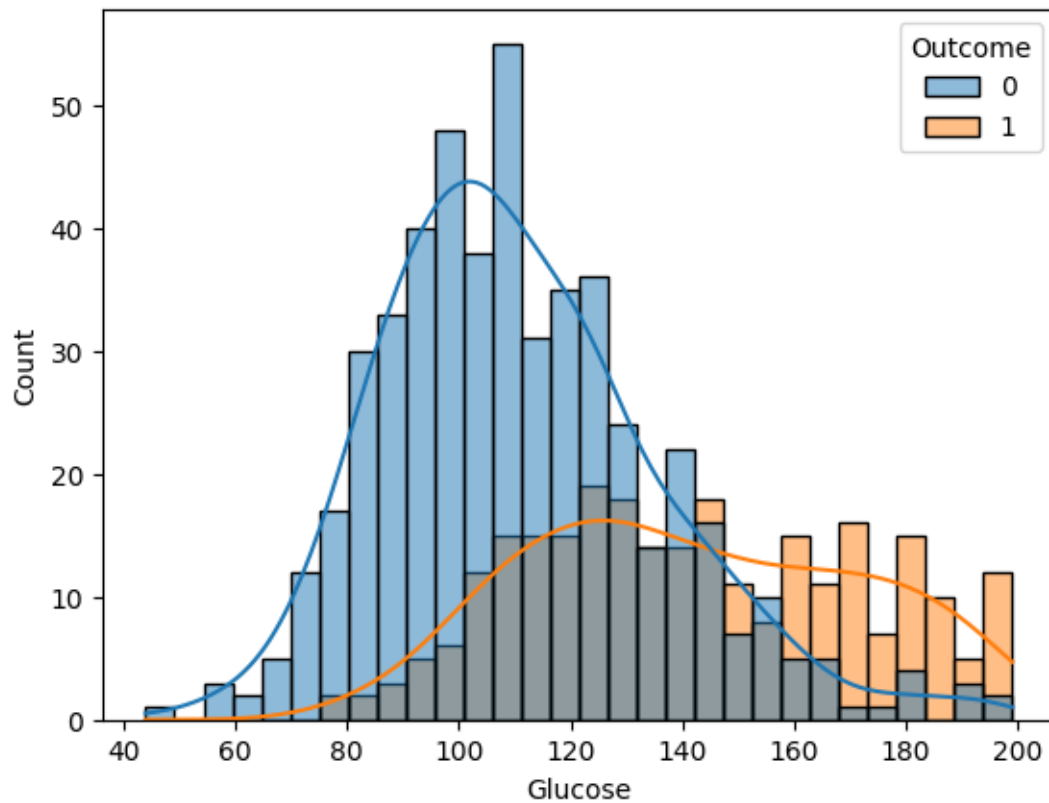
#### Discussão sobre Métricas de Outcome

- Normalmente observa-se **desbalanceamento moderado**
- Classe 0 (não diabético) costuma ser maior que Classe 1
- Isso impacta
  - Métricas de avaliação
  - Escolha de modelos
  - Possivelmente técnicas de balanceamento

```
sns.histplot(data=diabetes_data, x="Glucose", hue="Outcome", kde=True, bins=30)
```

```
plt.title="Distribuição de Glucose por Classe"
```

```
plt.show()
```



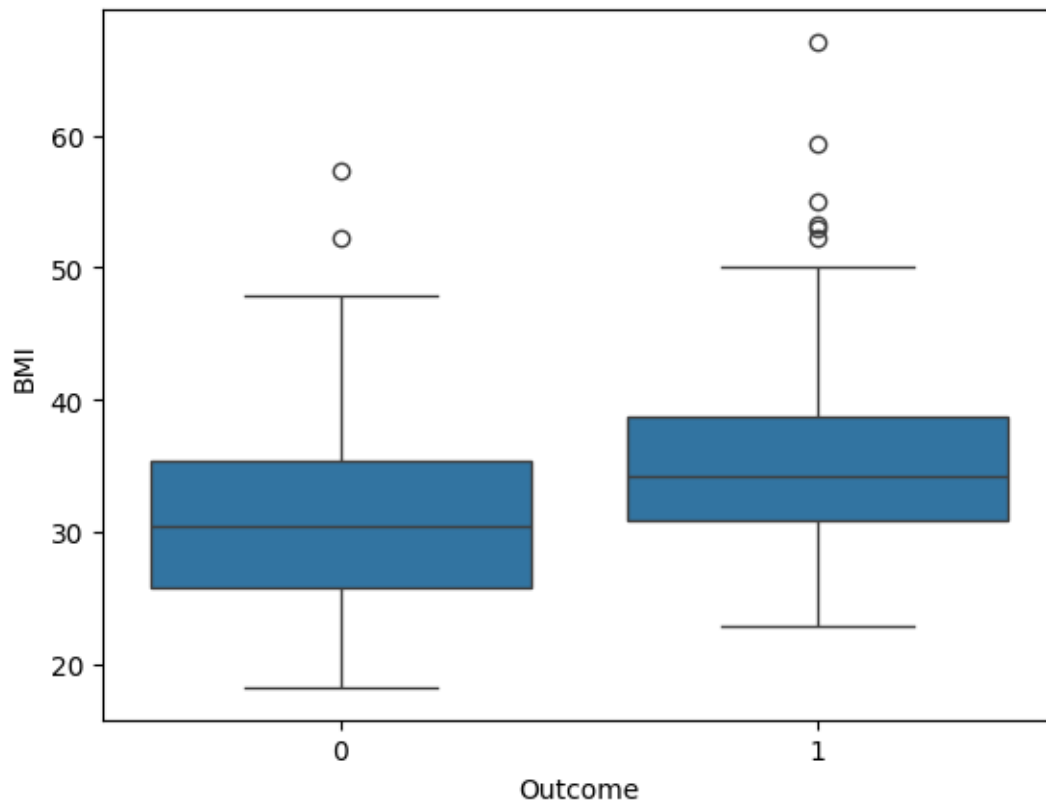
#### Discussão sobre Glucose por Classe

- Indivíduos com **Diabetes (Outcome = 1)** tendem a ter:
  - Médias mais altas de Glucose
  - Distribuição deslocada à direita
- Forte indicador preditivo

```
sns.boxplot(x="Outcome",y="BMI",data=diabetes_data)
```

```
plt.title="BMI por Classe"
```

```
plt.show()
```



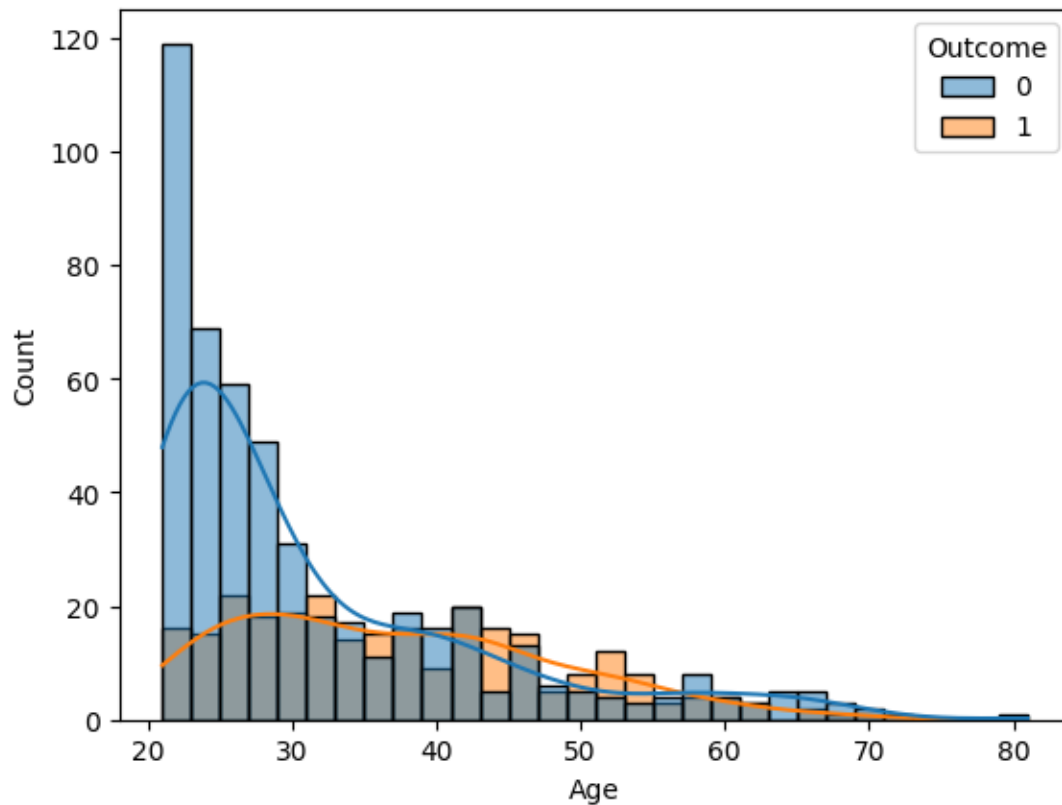
Discussão sobre BMI vs. Outcome

- Classe **diabética** costuma apresentar:
  - BMI médio mais alto
  - Maior variabilidade
- Relação positiva entre diabetes e obesidade é consistente com literatura médica

```
sns.histplot(data=diabetes_data, x="Age", hue="Outcome", kde=True, bins=30)
```

```
plt.title("Distribuição de Age por Classe")
```

```
plt.show()
```



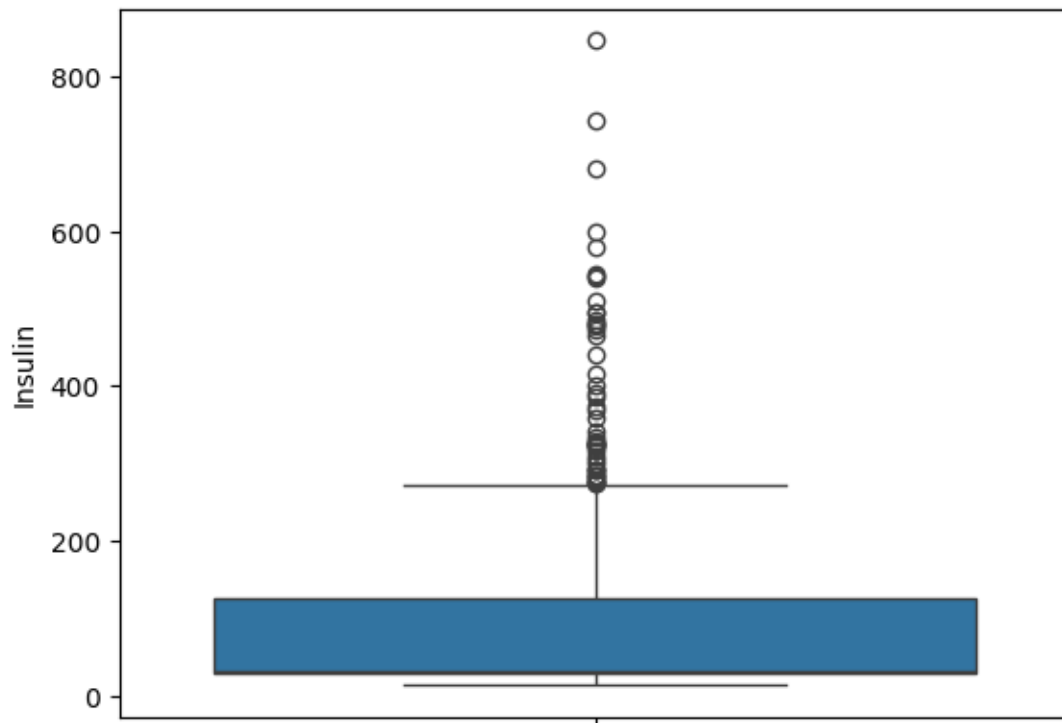
Discussão sobre Idade vs. Outcome

- Maior incidência de diabetes sobre idades mais altas
- Distribuição mais concentrada em faixas etárias superiores para Outcome = 1
- Idade deve ser considerada uma feature importante

```
sns.boxplot(y="Insulin",data=diabetes_data)
```

```
plt.title="Distribuição de Insulin"
```

```
plt.show()
```



Discussão sobre Insulina e Skin Thickness (atenção especial)

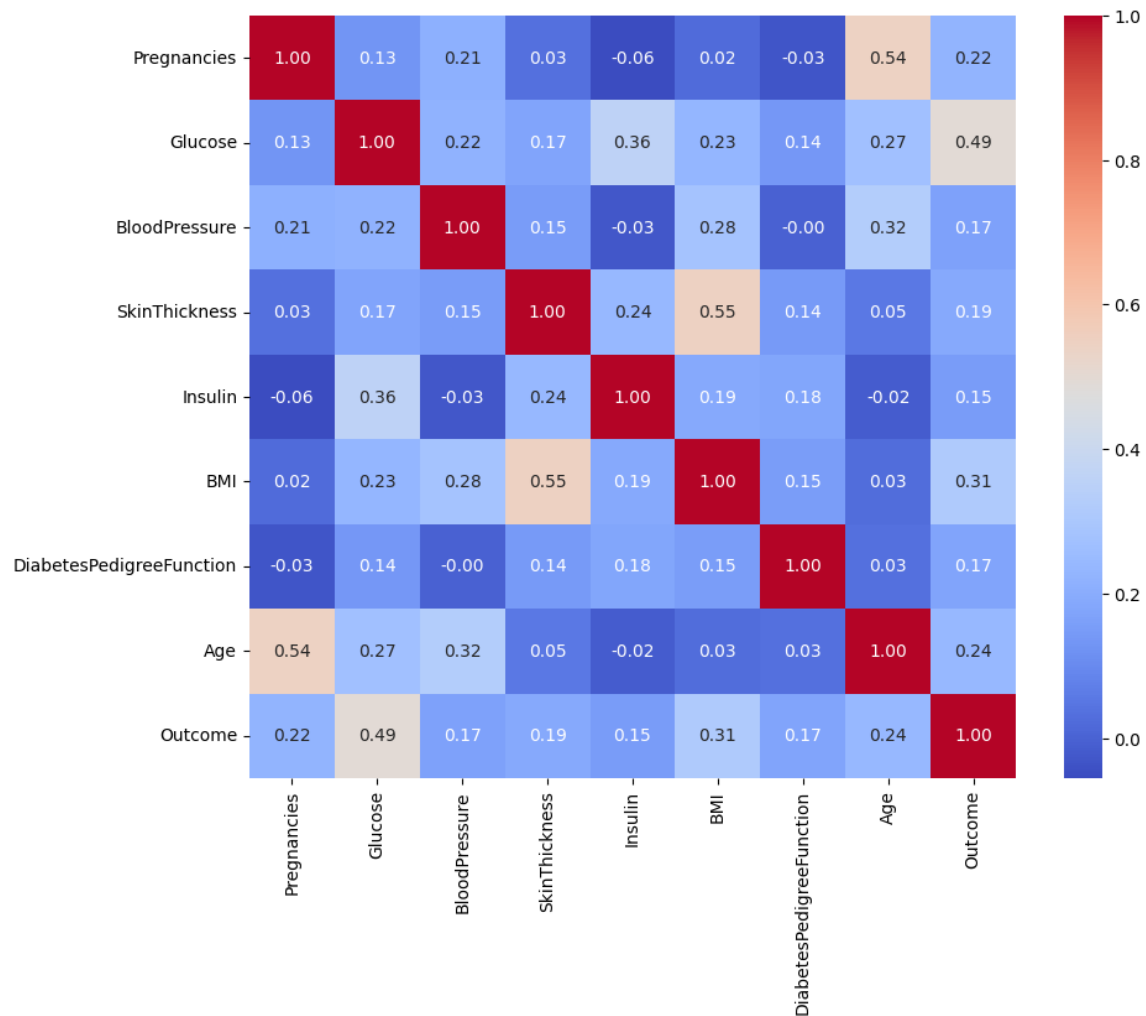
- Distribuição altamente assimétrica
- Presença de:
  - Muitos zeros
  - Outliers extremos
- Antes de usar no modelo:
  - Substituir zeros por NaN
  - Aplicar imputação
  - Possivelmente usar transformação logarítmica

```
plt.figure(figsize=(10,8))
```

```
sns.heatmap(diabetes_data.corr(), annot=True, cmap="coolwarm", fmt=".2f")
```

```
plt.title="Matriz de Correlação"
```

```
plt.show()
```



### Pontos relevantes

- Glucose costuma ter a maior correlação com Outcome
- BMI e Idade também mostram correlação positiva
- Pouca multicolinearidade severa

### Discussão Geral dos resultados

#### Principais insights

- Glucose é a feature mais informativa
  - Clara separação entre as classes
- BMI e Idade são fatores de risco importantes
  - Coerente com evidências clínicas

- Zeros representam dados ausentes
  - Devem ser tratados antes do treinamento
- Dataset é relativamente pequeno
  - Modelos simples podem funcionar bem
  - Validação cruzada é recomendada

## Apêndice h – Dockerfile

# Imagem base

FROM python:3.11-slim

# Variáveis de ambiente

ENV PYTHONDONTWRITEBYTECODE=1

ENV PYTHONUNBUFFERED=1

# Diretório de trabalho

WORKDIR /app

# Copiar dependências

COPY requirements.txt .

# Instalar dependências do sistema

RUN apt-get update && \

apt-get install -y --no-install-recommends build-essential && \

rm -rf /var/lib/apt/lists/\*

# Instalar dependências Python

RUN pip install --upgrade pip && \

pip install -r requirements.txt

# Copiar projeto



COPY ..

# Comando padrão

CMD ["python", "src/DiagnosisTool/train.py"]